

What Is a Shell Script?

A shell script is a file that contains one or more commands, which can be commands that you might type on a command line. The script file when executed (run) would run these commands *one after the other* like how one might have typed them on a command line one after the other sequentially.

```
jloka@jloka-VirtualBox:~/scripts$ chmod u+x script_01.sh
jloka@jloka-VirtualBox:~/scripts$ scr
screendump      script      scriptlive      scriptreplay
jloka@jloka-VirtualBox:~/scripts$ ./script_01.sh
Hello Bash World!
jloka@jloka-VirtualBox:~/scripts$ type test
test is a shell builtin
jloka@jloka-VirtualBox:~/scripts$ test
jloka@jloka-VirtualBox:~/scripts$ test --help
jloka@jloka-VirtualBox:~/scripts$
```

Naming the Script File One of the most common mistakes that beginners make is calling the trial script test. At first, it does make sense

The `type` command tells us what the shell will execute (and where this could be found if it is an external file) for any given command. In bash, ***type -a test*** will display all the commands that match the name test.

```
jloka@jloka-VirtualBox:~/scripts$ type -a test
test is a shell builtin
test is /usr/bin/test
test is /bin/test
```

Typically, Unix command names are as short as possible. They are often the first two consonants of a descriptive word, for example, mv for move or ls for list, or the first letters of a descriptive phrase, for example, *ps for process status* or *sed for stream editor*.

```
jloka@jloka-VirtualBox:~/scripts$ mv script_01.sh hw.sh
jloka@jloka-VirtualBox:~/scripts$ ls -la
total 12
drwxrwxr-x  2 jloka jloka 4096 Aug  3 22:51 .
drwxr-x--- 16 jloka jloka 4096 Aug  3 22:36 ..
-rwxrw-r--  1 jloka jloka  35 Aug  3 22:43 hw.sh
jloka@jloka-VirtualBox:~/scripts$ ./hw.sh
Hello Bash World!
jloka@jloka-VirtualBox:~/scripts$
```

Many shell programmers add a suffix such as *.sh* to indicate that the program is a *shell script*. The script does not require it, and it is used for programs that are being developed. One way is to add the suffix *-sh* to the name during development, and then *when the program is completed, the suffix can be removed*. The shell script then becomes another command and does not need to be distinguished from any other type of command.

“A parameter is an entity that stores values.” There are three types of parameters: *positional parameters*, *special parameters*, and *variables*.

- *Positional parameters* are arguments present on the command line, and they are referenced by a number.
- *Special parameters* are set by the shell to store information about aspects of its current state, such as *the number of arguments* and *the exit code* of the last command. *Their names are non-alphanumeric characters* (e.g., *, #, and _).
- *Variables* are identified by a name.

The **first positional argument** is **\$1**, and the second one is **\$2**, and so on.

The **first optional parameter** is **\$0**, where it is the full path to shell file.

```
#!/bin/bash
```

```
echo Hello Bash World!
```

```
mkdir -p $HOME/scripts/bpl/bin
```

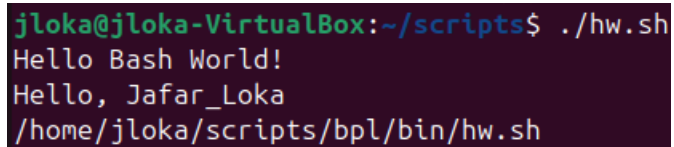
```
mkdir -p $HOME/scripts/bpl/scripts
```

```
echo echo Hello, \ $1 > $HOME/scripts/bpl/bin/hw.sh
```

```
echo echo \ $0 >> $HOME/scripts/bpl/bin/hw.sh
```

```
chmod +x $HOME/scripts/bpl/bin/hw.sh
```

```
$HOME/scripts/bpl/bin/hw.sh Jafar_Loka
```



```
jloka@jloka-VirtualBox:~/scripts$ ./hw.sh
Hello Bash World!
Hello, Jafar_Loka
/home/jloka/scripts/bpl/bin/hw.sh
```

The original Bourne shell could only address up to *nine positional parameters*. If a script were to use **\$10**, it would be interpreted as **\$1** followed by a literal **0**. To be able to run older scripts, bash maintains this behavior. To access positional parameters greater than **9**, the number must be enclosed in braces like **\${15}**.

The script is passed to the parameters that can be accessed via their positions, *\$0, \$1, \$2, and so on*. There exists a function *shift N*, which moves the positional parameters by N positions. If we ran *shift* (the default value of N is 1), then *\$0* would be discarded and *\$1* would become *\$0*, *\$2* would become *\$1*, and so on. They would all be shifted by *1 position*. There are some very clever and simple uses of *shift* to iterate through a list of parameters of unknown length.

Note: The *shift function* is *destructive*; that is, the parameters discarded are gone and cannot be retrieved again.

\$# → Special Parameter to the number of positional arguments given.

\$ && @\$* → Expand the list of parameters that given to shell (array of values that pass with shell).

\$\$ → Prints the PID.

\$? → Set to the exit code of the last executed command.

\$_ → Set to the last argument to that command.

\$! → Contains the PID of the last command executed in the background.

\$- → Set to the options flag currently in effect

```
#!/bin/bash
```

```
echo Hello Bash World!
```

```
mkdir -p $HOME/scripts/bpl/bin
```

```
mkdir -p $HOME/scripts/bpl/scripts
```

```
echo echo Hello, \${1} > $HOME/scripts/bpl/bin/hw.sh
```

```
echo echo \${0}, \${#}, \${*}, \${@} >> $HOME/scripts/bpl/bin/hw.sh
```

```
echo echo \${$} >> $HOME/scripts/bpl/bin/hw.sh
```

```
echo echo \${?} >> $HOME/scripts/bpl/bin/hw.sh
```

```
echo echo \${$-} >> $HOME/scripts/bpl/bin/hw.sh
```

```
echo echo \${!} >> $HOME/scripts/bpl/bin/hw.sh
```

```
chmod +x $HOME/scripts/bpl/bin/hw.sh
```

```
$HOME/scripts/bpl/bin/hw.sh Jafar_Loka Jafar_Loka_02 Jafar_Loka_03
```

```
echo \${$-}
```

```
jloka@jloka-VirtualBox:~/scripts$ ./hw.sh
Hello Bash World!
Hello, Jafar_Loka
Jafar_Loka
/home/jloka/scripts/bpl/bin/hw.sh, 3, Jafar_Loka Jafar_Loka_02 Jafar_Loka_03, Jafar_Loka Jafar_Loka_02 Jafar_Loka_03
4276
0
hB
hB
```

```
*****
```

Variables

A variable is a parameter denoted by a name; a name is a word containing only letters, numbers, or underscores and beginning with a letter or an underscore. Values can be assigned to variables as such:

name=value

Note: bash is particular in the way it handles spacing; there are no spaces before the = and none after as well. If you have spaces, the command will not work.

Many variables are set by the shell itself, including the three we have already seen: **HOME**, **PWD**, and **PATH**. With the two minor exceptions of **auto_resume** and **histchars**, all the variables set by the shell are all in uppercase letters.

The default newline can be suppressed by using the option **-n**.

```
$          echo The quick brown fox
The quick brown fox
```

The default newline can be suppressed by using the option **-n**.

```
$          echo -n No newline
No newline$
```

The other way of escaping and suppressing the newline character was the use of the escape sequence `/c`, which would not work on many of the bash shells. However, we can use the option `-e`, which tells echo to recognize all the provided escape sequences.

```
$          echo "No Newline\c"
No newline
$          echo -e "No newline\c"
No newline$
```

Note: `bash` has an `xpg_echo` option (*XPG stands for X/Open Portability Guide*, a specification for Unix systems) that makes echo behave like that other version. This can be turned on or off while in the shell (using `shopt -s xpg_echo` either at the command line or in a script), or it can be turned on when the shell is compiled.

Escape Sequences

Escape sequences are single letters preceded by a backslash:

- `\a`: Alert (bell)
- `\b`: Backspace
- `\e`: Escape character
- `\f`: Form feed
- `\n`: Newline
- `\r`: Carriage return
- `\t`: Horizontal tab
- `\v`: Vertical tab
- `\\`: Backslash
- `\nnn`: A character specified by one to three octal digits
- `\xHH`: A character specified by one or two hexadecimal digits

The backslashes must be protected from the shell by quotes or another backslash:

```
$          printf "Q\t\141\n\x42\n"
Q   a
B
```

To store the result of operation in variable we use -v

```
#!/bin/bash

printf -v num "%04d" 4
printf "%s\n" "$num"

*****
```

When a command *reads a character or a line*, it reads from the standard input stream, which is the keyboard.

When it prints information, it is sent to the standard output, the monitor.

The third stream, standard error, is also connected to the monitor; as the name implies, it is used for error messages.

These streams are referred to by numbers, *called file descriptors (FDs)*.

These are 0, 1, and 2, respectively. The stream names are also often contracted to stdin, stdout, and stderr.

I/O streams can be redirected to (or from) a file or into a pipeline.

```
*****
```

Redirecting standard output *does not redirect standard error*. Error messages will still be displayed on the monitor. To send the error messages to a file — in other words, *to redirect FD2* — the redirection operator is preceded by the *FD*.

```
*****
```

Both standard output and standard error can be redirected on the same line.

```
#!/bin/bash

printf "%s\n%v\n" OK? JLoka-Error-Value > output.txt 2> err_file.txt

*****
```

In this case, the error message is going to a special file, /dev/null. Sometimes called the bit bucket, anything written to it is discarded. *\$ printf '%s\n%v\n' OK? Oops! 2>/dev/null*

```
*****
```


Instead of *sending output to a file*, it can be *redirected to another I/O stream* by using `>&N`, where *N* is the *number of the file descriptor*.

In this case both output && error will be redirected to file1.txt

```
#!/bin/bash
```

```
printf "%s\n%\n" JLoka-ITE-Developer Test-01 > file1.txt 2>& 1
*****
```

```
# here we first redirect to standard output that is screen, then redirect
the success to file2.txt
```

```
printf "%s\n%\n" JLoka-01 Test-01 2>&1 > file2.txt
*****
```

- bash has also a *nonstandard syntax* for redirecting both *standard output and standard error* to the same place: `&> FILE`
- To append both standard output and standard error to FILE, use this: `&>> FILE`

```
*****
```

We can use the *exec command to redirect the I/O streams* for the rest of the script or until it's changed again.

```
exec 1>tempfile
```

```
exec 0<datafile
```

```
exec 2> errorfile
```

All standard output will now go to the file *tempfile*, *input will be read from datafile*, and *error messages* will go to *errorfile* without having to specify it for every command.

```
*****
```

```
read_data.sh
1  #!/bin/bash
2
3  echo "Enter 3 values at least"
4
5  read j1 j2 j3
6
7  echo "j1 is: $j1"
8  echo "j2 is: $j2"
9  echo "j3 is: $j3"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● jloka@jloka-VirtualBox:~/scripts\$./redirect to same.sh
./redirect to same.sh: line 5: printf: 'v': invalid format character

● jloka@jloka-VirtualBox:~/scripts\$ git add . && git commit -m "redirect in different way"
[main 30eef22] redirect in different way
2 files changed, 5 insertions(+), 1 deletion(-)
create mode 100644 file2.txt

● jloka@jloka-VirtualBox:~/scripts\$ chmod +x read_data.sh

● jloka@jloka-VirtualBox:~/scripts\$./read_data.sh
Enter 3 values at least
1 2 3 4 5 6 7 8 9

● jloka@jloka-VirtualBox:~/scripts\$./read_data.sh
Enter 3 values at least
1 2 3 4 5 6 7 8 9
j1 is: 1
j2 is: 2
j3 is: 3 4 5 6 7 8 9

○ jloka@jloka-VirtualBox:~/scripts\$ █